Data Analytics using R and Python

Introduction to Machine Learning & ML Libraries in R

Aayush Sharma

Department of CSE & IT

Lecture Outline

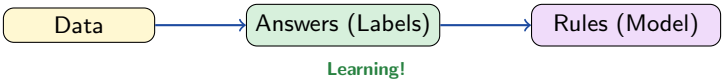
1. What is Machine Learning?
2. Supervised Learning
3. Unsupervised Learning
4. The `caret` Package – Deep Dive
5. The `tidymodels` Framework
6. Model Evaluation & Practical R Patterns
7. References

From Rules to Learning – Why ML Exists

The classical programming model:



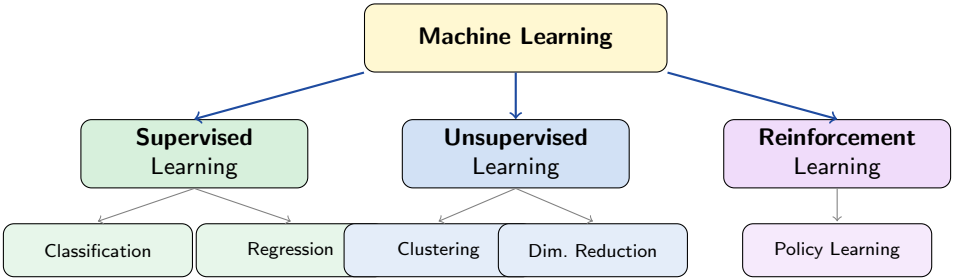
The machine learning model:



Definition

Machine Learning is the field of study that gives computers the ability to **learn from data** without being explicitly programmed for every scenario. A computer program is said to learn from experience E with respect to some task T and performance measure P if its performance on T, as measured by P, *improves* with experience E. *– Tom Mitchell, 1997*

The Three Pillars of ML – A Map



This lecture covers: Supervised and Unsupervised learning, with deep focus on implementing each in R using `caret`, `tidymodels`, and related packages.

Core Terminology – The Vocabulary You Must Know

Data Concepts

- **Feature / Predictor (X):** An input variable (column) used to make predictions.
- **Target / Label (y):** The output variable we want to predict.
- **Observation / Instance:** One row in the dataset.
- **Training set:** Data the model learns from.
- **Test set:** Held-out data to evaluate the final model.
- **Validation set:** Used during training to tune hyperparameters.

Model Concepts

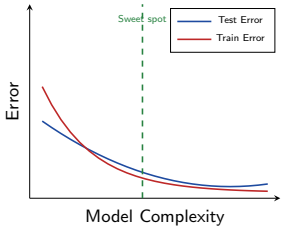
- **Hyperparameter:** A setting chosen before training (e.g. number of neighbours in kNN).
- **Parameter:** Learned from data during training (e.g. regression coefficients).
- **Inference / Prediction:** Applying a trained model to new data.
- **Generalisation:** How well a model performs on unseen data.
- **Pipeline:** A sequence of preprocessing + modelling steps.

The Bias–Variance Trade-off – The Central Problem of ML

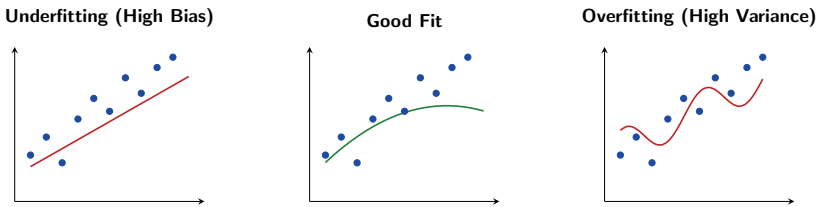
Decomposition of Error

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

- **High Bias (Underfitting):** Model is too simple; fails to capture the true pattern. Both training and test error are high.
- **High Variance (Overfitting):** Model is too complex; memorises the training data including noise. Training error is low but test error is high.
- **Sweet spot:** A model with just enough complexity to generalise well.

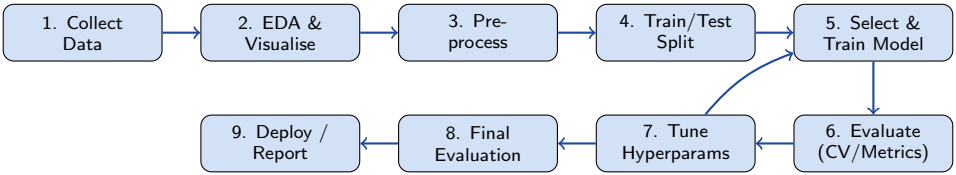


Overfitting vs Underfitting – Visual Intuition



In R: Overfitting can be detected when `train` accuracy $\gg$ `test` accuracy. Cross-validation (built into `caret`/`tidymodels`) is the standard defence.

The ML Workflow – From Raw Data to Deployed Model



Key R packages for each step:

Step	R Tools
EDA & Visualise	`ggplot2`, `GGally`, `skimr`, `DataExplorer`
Preprocess	`recipes` (tidymodels), `caret::preProcess`
Train/Tune/Evaluate	`caret`, `tidymodels` (`parsnip` + `tune` + `rsample`)
Specialised models	`randomForest`, `e1071` (SVM), `class` (kNN), `rpart`

Supervised Learning – The Setup

Definition

In **supervised learning** we are given a labelled dataset $\{(x_i, y_i)\}_{i=1}^{n}$ and the goal is to learn a function f such that $f(x) \approx y$ for new, unseen inputs x.

Classification (discrete y)

- Binary: spam/not-spam, disease/healthy
- Multi-class: digit recognition (0–9), species
- Output: a *class label* (and often a probability)

Regression (continuous y)

- House price prediction
- Temperature forecasting
- Output: a *real number*

The key assumption

Training data is an *i.i.d. sample* from the same distribution as future data. If this breaks down (data drift, concept drift), the model will fail silently – a critical real-world concern.

Algorithms we cover:
Linear Regression, Logistic Regression, k-NN, SVM, Decision Trees, Random Forests.

Linear Regression – Learning a Line from Data

Model:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = \mathbf{x}^\top \boldsymbol{\beta}$$

Learning objective – Ordinary Least Squares (OLS):

$$\min_{\boldsymbol{\beta}} \ \text{RSS} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Closed-form solution: $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

Key assumptions (LINE):

- **L**inearity: $E[y \mid x]$ is linear in x.
- **I**ndependence: residuals are independent.
- **N**ormality: residuals $\sim \mathcal{N}(0, \sigma^2)$.
- **E**qual variance (homoscedasticity).

Evaluation metrics:

- $\text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$
- $R^2 = 1 - \dfrac{\text{RSS}}{\text{TSS}}$
- Adjusted R^2 penalises extra predictors.

Linear Regression in R – lm() and caret

```r
# ============================================================
# Linear Regression: base R vs caret
# ============================================================
data(mtcars)

# --- Base R: fit and inspect ---
model_lm <- lm(mpg ~ wt + hp + cyl, data = mtcars)
summary(model_lm)           # coefficients, R^2, p-values
plot(model_lm, which = 1)   # residuals vs fitted (check homoscedasticity)
plot(model_lm, which = 2)   # Q-Q plot (check normality)

# --- caret: cross-validated linear regression ---
library(caret)
set.seed(42)
ctrl <- trainControl(method = "cv", number = 10)    # 10-fold CV
model_cv <- train(mpg ~ wt + hp + cyl,
                  data   = mtcars,
                  method = "lm",
                  trControl = ctrl)
print(model_cv)             # CV RMSE, R^2
varImp(model_cv)            # which predictors matter most?

# Predict on new data
new_car <- data.frame(wt = 3.2, hp = 120, cyl = 6)
predict(model_cv, newdata = new_car)
```

Logistic Regression – Classification via Probability

Problem: Linear regression can predict values outside $[0, 1]$, which is invalid for probabilities.
Solution: Squash the linear predictor through the **sigmoid function**:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{x}^\top \boldsymbol{\beta}) = \frac{1}{1 + e^{-\mathbf{x}^\top \boldsymbol{\beta}}}$$

Decision rule: Predict class 1 if $P(y = 1 \mid x) \geq 0.5$ (threshold is tunable).

Interpreting coefficients:
A unit increase in x_j multiplies the *odds* by e^{β_j}.

$$\text{odds} = \frac{P(y = 1)}{P(y = 0)}$$

If $\beta_j = 0.7$: each extra unit of x_j increases the odds by $e^{0.7} \approx 2\times$.

Fitting – Maximum Likelihood:
Minimise the *log-loss* (binary cross-entropy):

$$\mathcal{L} = -\sum_i \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

No closed form; solved iteratively (IRLS / gradient descent).

Logistic Regression in R

```r
# ============================================================
# Logistic Regression with caret
# ============================================================
library(caret)
data(PimaIndiansDiabetes, package = "mlbench")

set.seed(7)
# Stratified 80/20 split (preserves class balance)
idx   <- createDataPartition(PimaIndiansDiabetes$diabetes,
                             p = 0.8, list = FALSE)
train <- PimaIndiansDiabetes[ idx, ]
test  <- PimaIndiansDiabetes[-idx, ]

# Train with repeated 5-fold CV
ctrl <- trainControl(method       = "repeatedcv",
                     number       = 5,
                     repeats      = 3,
                     classProbs   = TRUE,          # needed for ROC
                     summaryFunction = twoClassSummary)

model_log <- train(diabetes ~ .,
                   data      = train,
                   method    = "glm",
                   family    = "binomial",
                   metric    = "ROC",              # optimise AUC
                   trControl = ctrl)
```

Understanding the Confusion Matrix

For a binary classifier with classes **Positive** (+) and **Negative** (−):

		Actual +	**Actual** −
Predicted	+	**TP**	**FP**
	−	**FN**	**TN**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{AUC-ROC} \in [0.5, 1]$$

Key insight: Accuracy is misleading on imbalanced data (e.g. 99% non-fraud means a classifier that always predicts non-fraud gets 99% accuracy but catches 0 frauds). **Always report Precision, Recall, and AUC.**

k-Nearest Neighbours (k-NN) – Learning by Analogy

Algorithm (no explicit training phase!)

To classify a new point x_{new}:

1. Compute the distance from x_{new} to every training point (Euclidean by default).
2. Find the k nearest training points.
3. Return the *majority class* among those k neighbours (classification) or their *mean* (regression).

Effect of k:

- Small k (e.g. $k = 1$): very flexible, high variance, jagged decision boundary.
- Large k: smooth boundary, higher bias.
- **Choosing k:** use cross-validation; odd k avoids ties in binary problems.

Critical pre-processing step:
kNN uses distances, so features must be on the *same scale*. **Always standardise** (zero mean, unit variance) before kNN.

In R

`preProcess(method = c("center","scale"))` then `method = "knn"` in `train()`.

k-NN in R with caret – Tuning k via CV

```r
# ============================================================
# kNN with automatic k-tuning via cross-validation
# ============================================================
library(caret)
data(iris)

set.seed(100)
idx   <- createDataPartition(iris$Species, p = 0.75, list = FALSE)
train <- iris[ idx, ]
test  <- iris[-idx, ]

# Pre-process: centre and scale (CRITICAL for kNN)
ctrl <- trainControl(method = "cv", number = 10)

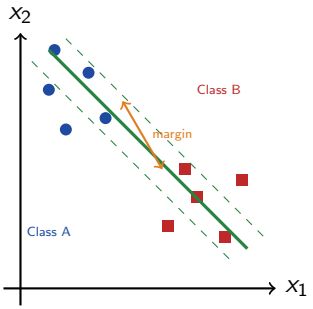
model_knn <- train(Species ~ .,
                   data       = train,
                   method     = "knn",
                   trControl  = ctrl,
                   preProcess = c("center", "scale"),
                   tuneLength = 15)   # try k = 5,7,9,...,29

# Plot accuracy vs k
plot(model_knn, main = "kNN: Accuracy vs k")

# Best k chosen automatically
cat("Best k:", model_knn$bestTune$k, "\n")
```

Support Vector Machines – Maximum Margin Classifiers

Core idea: Find the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that **maximises the margin** between the two classes.



Optimisation problem:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top x_i + b) \geq 1$$

Kernel trick: for non-linearly separable data, map to a higher-dimensional space using a kernel $K(x_i, x_j)$:

- Linear: $K = x_i^\top x_j$
- RBF: $K = e^{-\gamma\|x_i - x_j\|^2}$
- Polynomial: $K = (x_i^\top x_j + c)^d$

In R: `e1071::svm()`, method = `"svmRadial"` in caret.

SVM in R – Tuning C and γ

```
# ============================================================
# SVM with RBF kernel -- tune C and gamma via grid search
# ============================================================
library(caret)
library(e1071)
data(iris)

set.seed(42)
idx   <- createDataPartition(iris$Species, p = 0.8, list = FALSE)
train <- iris[ idx, ]
test  <- iris[-idx, ]

ctrl <- trainControl(method = "cv", number = 5)

# Define the tuning grid for C (cost) and sigma (= gamma in RBF)
grid <- expand.grid(C     = c(0.1, 1, 10, 100),
                    sigma = c(0.001, 0.01, 0.1, 1))

model_svm <- train(Species ~ .,
                   data      = train,
                   method    = "svmRadial",
                   trControl = ctrl,
                   tuneGrid  = grid,
                   preProcess = c("center","scale"))

# Best hyperparameters
```

Random Forests – Wisdom of the Crowd

Random Forests build many decision trees and combine their predictions:

1. Draw B bootstrap samples from the training data.
2. For each sample, grow a decision tree. At each split, consider only a **random subset of m features** (typically $m \approx \sqrt{p}$ for classification).
3. Aggregate: **majority vote** for classification, **mean** for regression.

Why it works:

- Bootstrap + random features *decorrelate* the trees.
- Averaging decorrelated noisy estimators reduces variance without increasing bias much.
- Resistant to overfitting as B grows.

Variable Importance (two measures):

- **Mean Decrease in Accuracy**: permute feature j, measure accuracy drop.
- **Mean Decrease in Gini**: total Gini decrease from splits on feature j.

Key hyperparameters: `ntree` (number of trees, usually ≥ 500) and `mtry` (features per split).

Random Forest in R – Full Pipeline

```
# ============================================================
# Random Forest with caret + variable importance
# ============================================================
library(caret)
library(randomForest)
data(PimaIndiansDiabetes, package = "mlbench")

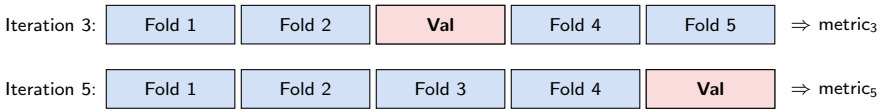
set.seed(7)
idx   <- createDataPartition(PimaIndiansDiabetes$diabetes,
                             p = 0.8, list = FALSE)
train <- PimaIndiansDiabetes[ idx, ]
test  <- PimaIndiansDiabetes[-idx, ]

ctrl <- trainControl(method = "cv", number = 5,
                     classProbs = TRUE,
                     summaryFunction = twoClassSummary)

# Tune mtry (number of features at each split)
model_rf <- train(diabetes ~ .,
                  data      = train,
                  method    = "rf",
                  metric    = "ROC",
                  trControl = ctrl,
                  tuneLength = 5,          # try 5 values of mtry
                  ntree      = 500)
```

Cross-Validation – The Right Way to Estimate Generalisation

Why not just use train/test split? A single split gives a noisy estimate of performance – lucky or unlucky splits can mislead.



Final CV score $= \overline{\text{metric}_1, \ldots, \text{metric}_k}$

Variants:

- k-fold (typically $k = 5$ or 10)
- Stratified k-fold (preserves class balance)
- Leave-One-Out (LOOCV): $k = n$
- Repeated k-fold: run k-fold r times, average

In R (`caret`):

- `method = "cv"` – standard k-fold
- `method = "repeatedcv"` – repeated k-fold
- `method = "LOOCV"`
- `method = "boot"` – bootstrap resampling

Supervised Learning – Puzzle

★ **Puzzle:** A logistic regression model is trained on a medical dataset to detect a rare disease (2% prevalence). On the test set of 1000 patients, it predicts *all patients as healthy*. What is its accuracy, precision, recall, and F_1 score? What does this tell you about choosing the right metric?

Solution:

- Out of 1000: 20 diseased (+), 980 healthy (−). Model predicts all 1000 as healthy.
- $TP = 0,\ FP = 0,\ FN = 20,\ TN = 980$
- Accuracy $= \dfrac{0 + 980}{1000} =$ **98%** – looks great, but useless!
- Precision $= \dfrac{0}{0 + 0} =$ undefined
- Recall $= \dfrac{0}{0 + 20} =$ **0%** – catches *no* sick patients.
- $F_1 = 0$

Lesson: On imbalanced data, **accuracy is a trap**. Always use Recall, F_1, and AUC. In R, use `metric = "ROC"` or `metric = "F"` in `trainControl`.

Unsupervised Learning – Finding Structure Without Labels

Definition

In **unsupervised learning** there are no labels y. The goal is to discover **hidden structure** in the input data $\{x_i\}_{i=1}^n$ alone.

Main tasks:

- **Clustering:** Group similar observations together.
- **Dimensionality Reduction:** Compress data to fewer dimensions while preserving structure (PCA, t-SNE, UMAP).
- **Density Estimation:** Learn the data distribution.
- **Anomaly Detection:** Find outliers/unusual points.

Challenge: There is no ground truth to evaluate against – evaluation is harder than supervised learning.

Common metrics:

- Within-cluster sum of squares (WCSS)
- Silhouette score: $s(i) = \dfrac{b(i) - a(i)}{\max(a(i), b(i))}$
- Davies-Bouldin index

k-Means Clustering – Algorithm and Geometry

Lloyd's Algorithm

1. Choose k initial centroids (randomly or with k-means++ seeding).
2. **Assignment step:** Assign each point to its nearest centroid.
3. **Update step:** Recompute each centroid as the mean of its assigned points.
4. Repeat steps 2–3 until centroids stop moving (convergence).

Objective (minimise):

$$\text{WCSS} = \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

This is NP-hard in general, but Lloyd's algorithm converges to a local minimum quickly.

Choosing k – the Elbow Method:
Plot WCSS against k. Pick the k where additional clusters give diminishing returns (the "elbow").

Limitations:

- Assumes spherical, equal-sized clusters.
- Sensitive to outliers and initialisation.
- Must specify k in advance.

k-Means in R – Elbow Method & Silhouette

```r
# ============================================================
# k-Means: elbow plot + silhouette analysis
# ============================================================
library(ggplot2)
data(iris)
X <- scale(iris[, 1:4])    # ALWAYS scale before clustering

# --- Elbow method: WCSS for k = 1..10 ---
wcss <- sapply(1:10, function(k) {
  kmeans(X, centers = k, nstart = 25)$tot.withinss
})
ggplot(data.frame(k = 1:10, wcss), aes(k, wcss)) +
  geom_line() + geom_point() +
  labs(title = "Elbow Plot", x = "Number of clusters k", y = "WCSS")

# --- Silhouette analysis ---
library(cluster)
sil <- sapply(2:8, function(k) {
  km  <- kmeans(X, centers = k, nstart = 25)
  ss  <- silhouette(km$cluster, dist(X))
  mean(ss[, 3])                # average silhouette width
})
cat("Best k by silhouette:", which.max(sil) + 1, "\n")

# --- Final model ---
km_final <- kmeans(X, centers = 3, nstart = 25)
```

Hierarchical Clustering – Building a Dendrogram

Agglomerative (bottom-up) approach:

1. Start: each point is its own cluster.
2. Merge the two *closest* clusters.
3. Repeat until one big cluster remains.
4. Cut the dendrogram at the desired height to get k clusters.

Linkage methods (how "distance between clusters" is defined):

Method	Distance	Cluster shape
Single linkage	Min pairwise distance	Elongated (chaining)
Complete linkage	Max pairwise distance	Compact, spherical
Average linkage	Mean pairwise distance	Balanced
Ward's method	Min increase in WCSS	**Usually best**

Advantage over k-means: No need to specify k upfront; the dendrogram shows the full hierarchy and natural number of clusters.

Hierarchical Clustering in R

```r
# ============================================================
# Hierarchical Clustering with dendrogram
# ============================================================
data(iris)
X <- scale(iris[, 1:4])

# Compute distance matrix and cluster
d      <- dist(X, method = "euclidean")
hc     <- hclust(d, method = "ward.D2")    # Ward's linkage

# Plot the dendrogram
plot(hc, labels = FALSE, hang = -1,
     main = "Hierarchical Clustering Dendrogram (Ward.D2)")
rect.hclust(hc, k = 3, border = c("red","blue","green"))

# Cut tree to get 3 clusters
clusters <- cutree(hc, k = 3)
table(iris$Species, clusters)

# ---- Compare k-means vs hierarchical ----
# How different are their assignments?
km3  <- kmeans(X, centers = 3, nstart = 25)$cluster
hc3  <- cutree(hc, k = 3)
# Use adjustedRandIndex to measure agreement (1 = identical)
library(mclust)
adjustedRandIndex(km3, hc3)
```

PCA – Dimensionality Reduction Before Modelling

Principal Component Analysis (PCA) finds new axes (principal components) that capture maximum variance:

$$\text{PC}_j = \mathbf{v}_j^\top \mathbf{x}, \quad \mathbf{v}_j = j\text{-th eigenvector of Cov}(\mathbf{X})$$

Why use PCA before ML?

- Removes *multicollinearity* (correlated features confuse regression).
- Reduces dimensionality $\Rightarrow$ faster training, less overfitting.
- Enables 2D/3D visualisation of high-dimensional data.

How many PCs? Keep components that explain a cumulative variance threshold (e.g. 95%).

In R with caret/recipes

- Base R: `prcomp(X, scale. = TRUE)`
- caret: `preProcess(method = "pca", thresh = 0.95)`
- recipes: `step_pca(all_numeric(), threshold = 0.95)`

PCA is applied only to training data; the same rotation is then applied to test data – **never fit PCA on test data**.

Unsupervised Learning – Puzzle

★ **Puzzle:** You run k-means with $k = 3$ on a dataset and get WCSS $= 120$. You then run it with $k = 4$ and get WCSS $= 119$. A student argues: "$k = 4$ is obviously better because WCSS is lower." What is wrong with this reasoning? How would you properly choose between $k = 3$ and $k = 4$?

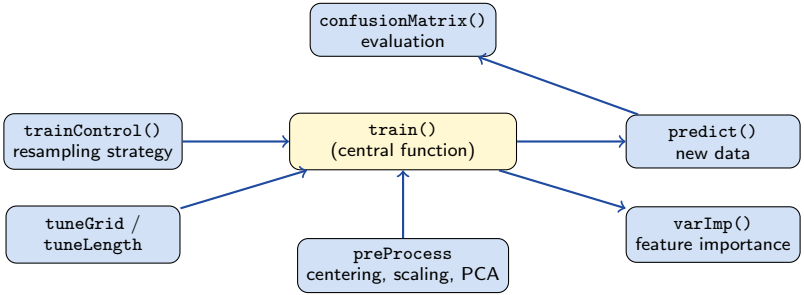
Solution:

- WCSS **always decreases** (or stays the same) as k increases – at $k = n$ every point is its own cluster and WCSS $= 0$. So a lower WCSS does not automatically mean a better model.
- The near-zero drop from 120 to 119 suggests the 4th cluster is not adding meaningful structure (no elbow).
- **Proper approach:**
  - Check the *elbow plot*: is the drop large relative to previous drops?
  - Compute the *silhouette score* for $k = 3$ and $k = 4$; higher is better.
  - In R: `silhouette(kmeans(X,3)$cluster, dist(X))` – compare average widths.
- **Lesson:** Comparing raw WCSS across different k is meaningless. Use relative change (elbow) or silhouette.

The caret Package – One Interface for 200+ Models

What is caret?

caret (Classification And REgression Training) by Max Kuhn provides a **unified interface** to over 200 ML algorithms in R, handling resampling, preprocessing, tuning, and evaluation in a consistent way.



caret – Pre-processing Pipeline in Depth

```
# ============================================================
# caret preProcess: centering, scaling, PCA, imputation
# ============================================================
library(caret)
data(iris)
X_train <- iris[1:100, 1:4]
X_test  <- iris[101:150, 1:4]

# Fit the pre-processor on TRAINING data only
pp <- preProcess(X_train,
                 method = c("center",     # subtract mean
                            "scale",      # divide by SD
                            "pca"),       # PCA on scaled data
                 thresh = 0.95)          # keep 95% variance

print(pp)   # shows how many PCs were kept

# Apply the SAME transformation to train AND test
X_train_pp <- predict(pp, X_train)
X_test_pp  <- predict(pp, X_test)

# IMPORTANT: the predict() call on pp applies the training
# mean/SD and PCA rotation to X_test -- it does NOT refit.
# This is the correct way to avoid data leakage.
cat("Original features:", ncol(X_train),
    "-> After PCA:", ncol(X_train_pp), "\n")
```

caret – Custom Metric and Resampling

```r
# ============================================================
# Custom summary function + comparing multiple models
# ============================================================
library(caret)
data(PimaIndiansDiabetes, package = "mlbench")
set.seed(42)
idx   <- createDataPartition(PimaIndiansDiabetes$diabetes,
                             p = 0.8, list = FALSE)
train <- PimaIndiansDiabetes[ idx, ]

ctrl <- trainControl(
  method          = "repeatedcv",
  number          = 5,
  repeats         = 3,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,  # ROC, Sens, Spec
  savePredictions = "final"           # needed for ROC curves
)

# Train three models with the SAME resampling splits (set.seed!)
m_log <- train(diabetes~., data=train, method="glm",
               family="binomial", metric="ROC", trControl=ctrl)
m_knn <- train(diabetes~., data=train, method="knn",
               metric="ROC", trControl=ctrl,
               preProcess=c("center","scale"), tuneLength=10)
m_rf  <- train(diabetes~., data=train, method="rf",
```

tidymodels – The Modern R ML Ecosystem

What is tidymodels?

tidymodels is a collection of packages built around tidy data principles. It replaces `caret` with a more modular, pipe-friendly design. Key packages:

Package	Role	Key functions
`rsample`	Splitting / resampling	`initial_split()`, `vfold_cv()`
`recipes`	Feature engineering	`recipe()`, `step_*()`
`parsnip`	Model specification	`logistic_reg()`, `rand_forest()`
`workflows`	Bundle recipe + model	`workflow()`, `add_recipe()`
`tune`	Hyperparameter tuning	`tune_grid()`, `select_best()`
`yardstick`	Metrics	`accuracy()`, `roc_auc()`

Philosophy: each step is explicit and composable. Preprocessing is defined as a `recipe`, the model as a `parsnip` spec, and they are combined into a `workflow` – ensuring no data leakage and easy reuse.

tidymodels – End-to-End Classification Workflow

```
library(tidymodels)
data(PimaIndiansDiabetes, package = "mlbench")

# 1. Split
set.seed(42)
split <- initial_split(PimaIndiansDiabetes, prop = 0.8,
                       strata = diabetes)
train <- training(split);   test <- testing(split)

# 2. Recipe (preprocessing)
rec <- recipe(diabetes ~ ., data = train) |>
  step_normalize(all_numeric_predictors()) |>
  step_corr(all_numeric_predictors(), threshold = 0.9)

# 3. Model specification
rf_spec <- rand_forest(mtry = tune(), trees = 500) |>
  set_engine("randomForest") |>
  set_mode("classification")

# 4. Workflow = recipe + model
wf <- workflow() |> add_recipe(rec) |> add_model(rf_spec)

# 5. Tune with 5-fold CV
folds  <- vfold_cv(train, v = 5, strata = diabetes)
grid   <- grid_regular(mtry(range = c(2, 7)), levels = 6)
tuned  <- tune_grid(wf, resamples = folds, grid = grid,
```

tidymodels – Finalise and Evaluate

```r
# (continuing from previous slide)

# 6. Select best mtry
best_params <- select_best(tuned, metric = "roc_auc")
final_wf    <- finalize_workflow(wf, best_params)

# 7. Final fit on full training set, evaluate on test set
final_fit   <- last_fit(final_wf, split)

# 8. Metrics on the test set (no manual predict() needed!)
collect_metrics(final_fit)
#  accuracy   ...
#  roc_auc    ...

# 9. Confusion matrix
collect_predictions(final_fit) |>
  conf_mat(truth = diabetes, estimate = .pred_class)

# 10. ROC curve
collect_predictions(final_fit) |>
  roc_curve(diabetes, .pred_pos) |>
  autoplot()

# Bonus: extract variable importance from the final model
final_wf |>
  fit(train) |>
```

Data Leakage – The Silent Killer of ML Models

Definition

Data leakage occurs when information from the **test set (or future)** inadvertently influences the training process, leading to over-optimistic performance estimates that do not generalise.

Common forms:

- **Preprocessing leakage:** Fitting a scaler or PCA on the entire dataset (train + test) before splitting. The test-set statistics leak into the scaler.
- **Target leakage:** Including a feature that is a direct consequence of the target (e.g. "claim paid" as a feature when predicting "fraud").
- **Temporal leakage:** Using future data to predict the past in time-series problems.

Prevention in R:

- Always fit `preProcess()` or `recipe()` on *training data only*, then `predict()` on test data.
- Use `workflow()` in tidymodels – it automatically applies the recipe *inside* each CV fold.
- In `caret`, `preProcess` inside `train()` is also fold-safe.

tidymodels / caret – Tricky Puzzle

★ **Puzzle:** A student writes the following R code to evaluate a kNN model. Identify **all** the bugs.

```
X_scaled <- scale(iris[,1:4])
set.seed(1)
idx <- sample(1:150, 120)
train_X <- X_scaled[idx,]; test_X <- X_scaled[-idx,]
train_y <- iris$Species[idx]; test_y <- iris$Species[-idx]
library(class)
pred <- knn(train_X, test_X, train_y, k = 1)
mean(pred == test_y)
```

Solution – three bugs:

1. **Scaling on full data:** `scale(iris[,1:4])` uses the mean/SD of all 150 rows including the test set. Should scale only on training rows, then apply same parameters to test.
2. $k = 1$ **without tuning:** $k = 1$ nearly always overfits. The correct k should be chosen via cross-validation on the training set.
3. **No stratification:** `sample()` ignores class balance. With 3 equal classes this is okay here, but `createDataPartition()` is the safe habit.

Hyperparameter Tuning Strategies

Grid Search (`tuneGrid` in caret)

- Exhaustively tries all combinations in a predefined grid.
- Guaranteed to find the best in the grid.
- Scales poorly: p parameters with g values each $\Rightarrow g^p$ evaluations.

Random Search (`search = "random"` in caret)

- Randomly samples n combinations.
- Often finds near-optimal results faster than grid search.
- Preferred when ≥ 3 hyperparameters.

Bayesian Optimisation (`tune_bayes()` in tidymodels)

- Builds a surrogate model of the performance surface.
- Sequentially proposes the most promising next point.
- Most sample-efficient; best for expensive models.

Rule of thumb

- 1–2 params: Grid search
- 3–5 params: Random search
- Slow models: Bayesian optimisation

Putting It All Together – Benchmark Script in R

```r
# ============================================================
# Full benchmark: compare 4 models on the same CV folds
# ============================================================
library(caret); library(mlbench)
data(Sonar)   # 60 features, 208 obs, binary: Mine vs Rock
set.seed(42)
ctrl <- trainControl(method="repeatedcv", number=10, repeats=3,
                     classProbs=TRUE,
                     summaryFunction=twoClassSummary,
                     savePredictions="final")
# Common preprocessing
pp <- c("center","scale")
m1 <- train(Class~., Sonar, method="glm",
            family="binomial", metric="ROC", trControl=ctrl)
m2 <- train(Class~., Sonar, method="knn",
            preProcess=pp, metric="ROC", trControl=ctrl,
            tuneLength=10)
m3 <- train(Class~., Sonar, method="svmRadial",
            preProcess=pp, metric="ROC", trControl=ctrl,
            tuneLength=8)
m4 <- train(Class~., Sonar, method="rf",
            metric="ROC", trControl=ctrl, tuneLength=5)

res <- resamples(list(LR=m1,KNN=m2,SVM=m3,RF=m4))
bwplot(res, metric="ROC")      # box-and-whisker comparison
summary(res)
```

Common R Pitfalls in ML – Know Before You Code

Mistake	Why it's wrong	Fix in R
`scale()` on all data before splitting	Leaks test statistics into scaler	`preProcess()` inside `train()` or `recipe()`
Using `set.seed()` inside a loop	Different seeds per iteration, not reproducible	Set once before `createDataPartition()`
Comparing models on different test splits	Apples to oranges	`resamples()` with same `trainControl` and shared seed
Tuning on test set	Optimistic bias, model selected to fit test noise	Use validation set or nested CV
Ignoring class imbalance	Misleading accuracy	`sampling = "up"` or `"smote"` in `trainControl`
Factor levels mismatch	`predict()` errors on new data	`levels(test$y) <- levels(train$y)`

Advanced Puzzle – Spot the Leakage

★ **Puzzle:** A data scientist builds a model to predict employee attrition. Their dataset has 30 features including `exit_interview_score`. After a 80/20 train-test split and 10-fold CV on the training set, they achieve AUC = 0.97 on the test set. Their manager is thrilled. What is almost certainly wrong?

Solution:

- **Target leakage via `exit_interview_score`:** An exit interview is only conducted when an employee *has already decided to leave*. This feature is a direct consequence of the outcome (attrition = 1) and is unavailable at prediction time.
- Including it gives the model a "cheat code" – it essentially predicts attrition from attrition.
- AUC = 0.97 is a red flag: real attrition models typically achieve AUC $\approx$ 0.75–0.85. Suspiciously high performance almost always signals leakage.
- **Fix:** Remove all features that are *caused by* or *known only after* the target event. In R: `select(-exit_interview_score)` before modelling.

Rule: If a feature would be unavailable at the time you make a real prediction, it must not be in the model.

Feature Engineering Tricks in R (recipes)

The recipes package makes feature engineering a reproducible, leakage-free pipeline:

Step function	What it does
step_normalize()	Zero mean, unit variance (centre + scale)
step_log()	Log-transform skewed numeric features
step_dummy()	One-hot encode categorical variables
step_impute_median()	Replace NA with median (from train)
step_corr()	Remove highly correlated predictors
step_nzv()	Remove near-zero variance features
step_pca()	PCA dimensionality reduction
step_interact()	Create interaction terms $x_1 \times x_2$
step_poly()	Add polynomial terms x^2, x^3
step_smote()	SMOTE oversampling for imbalanced data

Key property: Each step_*() learns its parameters (mean, SD, PCA rotation, etc.) from the **training fold only**, then applies them to validation/test data. This prevents data leakage automatically.

References & Further Reading

- Mitchell, T.M. (1997). *Machine Learning*. McGraw-Hill.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.). Springer.
- Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer.
- Kuhn, M., & Silge, J. (2022). *Tidy Modeling with R*. O'Reilly. *(Free online: tmwr.org)*
- Kabacoff, R.I. (2015). *R in Action* (2nd ed.). Manning.
- Kuhn, M. et al. `caret`: Classification and Regression Training. CRAN.
- Kuhn, M. et al. `tidymodels`: A collection of packages for modelling. *tidymodels.org*
- Wickham, H. & Grolemund, G. (2017). *R for Data Science*. O'Reilly. *(Free online: r4ds.had.co.nz)*

Thank You!

Questions?

> "All models are wrong, but some are useful."
>
> — George Box

Aayush Sharma • Department of CSE & IT